Topic: Machine Learning

Concepts: Supervised learning, linear regression, training/validation/testing splits, loss functions, gradient descent, hyperparameter tuning.

Concrete compressive strength depends on mix proportions and curing conditions. In this assignment, you will try to predict the concrete strength in MPa as a function of curing age. For this, you will implement a linear regression model, implementing the way ML models are implemented. You will understand how to split the data, use training dataset for training, use validation dataset for hyperparameter tuning, how selection of loss affects final result and evaluate the final model accuracy using test dataset. You will also implement gradient descent to obtain the optimal parameters.

Task 1:

Locate the file "concrete_strength_dataset.npz", which contains the dataset to be used for ML. The dataset has following variables:

Input: Concrete curing age in days (referred as ca)

Output: Concrete strength in MPa (referred as fc)

Unpack the variables and store them into appropriate variables (i.e. ca in variable ca, fc in variable fc). Plot curing age against compressive strength. Comment on any trends.

Task 2:

Now, you will split the data into 70% training set, 10% validation set and 20% testing set using random number generator with seed 2. Find out what is the total number of samples in the data and store it in variable n. Find the number of training and validation examples and store them in n_train and n_val variables respectively. Using these variables, split the data using the indices variable. Print the sample count in each set. You will use training set to estimate the model parameters, validation set for hyperparameter tuning, and test set for final validation.

Task 3:

Take Initial learning rate, model size and number of epochs as 0.01, 2 and 5000. Use these initial hyperparameters. Following is various model sizes prescribed:

- Model 1: $f_c = C_1 + C_2(ca)$, size = 2
- Model 2: $f_c = C_1 + C_2(ca) + C_3(ca)^2$, size = 3
- Model 3: $f_c = C_1 + C_2(ca) + C_3(ca)^2 + C_4(ca)^3$, size =4

Task 4:

Write down the equation for each example individually and then convert them into matrix form $\boldsymbol{y_{pred} = X\theta}$, with $\boldsymbol{X}$ being the design matrix and $\boldsymbol{\theta}$ containing the unknown model parameters. Calculate gradients with respect to each model parameter analytically.

Add the code to calculate y_pred, mse as loss function, gradient using error variable and update parameters (in variable theta) each epoch. Then, implement gradient descent for the model. Record training loss to assess convergence.

Task 5: Train initial hyperparameters and evaluate validation loss

Train the model using your initial learning rate and model size. Predict the validation set and report validation MSE and RMSE in the original MPa units. Add the code to plot loss vs number of epochs. Comment on whether training appears to converge.

Task 6: Train additional hyperparameter combinations and select the best

Add the code to achieve nine different combinations varying: a) three learning rates: 0.001, 0.01, 0.1 and b) three model sizes described. For each combination, report training and validation MSE. Tabulate the results and select the configuration with the lowest validation MSE. Do not use test results to choose the configuration.

Task 7: Determine test accuracy and draw inferences

We will now evaluate the selected configuration on the test set. Add the code to compute MSE, RMSE and MAE using fc_test_pred and fc_test variables. Add the code to plot measured versus predicted strength superimposed with the ideal y = x line. Discuss the effect of model size and learning rate, and validation-versus-test performance.

Note that:

MSE = mean squared error = $\left(\frac{1}{N}\right)\sum_i \left(fc_i - fc_{i_{pred}}\right)^2$

RMSE = root mean squared error= $\sqrt{\left(\frac{1}{N}\right)\sum_i \left(fc_i - fc_{i_{pred}}\right)^2}$

MAE = mean absolute error = $\left(\frac{1}{N}\right)\sum_i |fc_i - fc_{i_{pred}}|$